

Documentação Técnica - Sistema JES

Jogos Escolares Superiores do IFPB

Visão Geral

Objetivo do Sistema

O Sistema JES é uma aplicação web desenvolvida em Django para gerenciar os Jogos Escolares Superiores do IFPB, permitindo: - Gestão de atléticas (criação, edição, gerenciamento de atletas) - Cadastro de atletas vinculados a atléticas - Criação de times por modalidade esportiva - Gestão de cargos administrativos (Admin, Moderador, Capitão) - Controle de acesso baseado em permissões hierárquicas - Autenticação via matrícula do estudante

Tecnologias Utilizadas

- **Backend:** Django 6.0.2
- **Banco de Dados:** PostgreSQL (Supabase)
- **Autenticação:** Django Auth + Backend Customizado
- **Frontend:** HTML5 + CSS3 (templates Django)
- **Linguagem:** Python 3.8+

Características Principais

- ☒ Autenticação por matrícula (não por username)
- ☒ Sistema hierárquico de permissões
- ☒ Gestão completa de atléticas por capitães
- ☒ Criação de times por modalidade esportiva
- ☒ Senha padrão = matrícula (troca obrigatória no primeiro login)
- ☒ Validação forte de senhas

-  Interface moderna e responsiva
 -  Controle de acesso baseado em vínculo com atlética
-

Regras de Negócio

Entidades Principais

Atlética

- Organização esportiva estudantil vinculada a um campus e curso
- Possui um Capitão responsável pela gestão
- Pode ter múltiplos atletas cadastrados
- Pode ter múltiplos times em diferentes modalidades
- Informações: nome, campus, curso, capitão

Atleta

- Estudante do IFPB que participa dos jogos
- Identificado por matrícula única
- Vinculado obrigatoriamente a uma atlética
- Pode ou não ter cargo administrativo
- Pode participar de múltiplos times (modalidades diferentes)

Modalidade

- Esporte/competição dos jogos (ex: Futsal, Vôlei, Basquete, Xadrez)
- Define características do time (número de jogadores, tipo)
- Pode ter múltiplos times de diferentes atléticas

Time

- Equipe de uma atlética em uma modalidade específica
- Composto por atletas da mesma atlética
- Gerenciado pelo Capitão da atlética

- Informações: nome do time, modalidade, atlética, lista de atletas

Usuário (UserProfile)

- Atleta com cargo administrativo
- Possui email e senha
- Pode fazer login no sistema
- Tem permissões baseadas no cargo e vínculo com atlética

Associado

- Vínculo entre Atleta e UserProfile
- Garante que apenas atletas podem ter cargos

Hierarquia de Cargos

Admin (Nível 3)

- └─ Pode cadastrar atletas em qualquer atlética
- └─ Pode criar, editar e excluir qualquer atlética
- └─ Pode criar e gerenciar times de qualquer atlética
- └─ Pode dar cargos: Capitão e Moderador
- └─ Pode remover qualquer cargo
- └─ Acessa estatísticas globais do sistema
- └─ Sem restrição de atlética

Moderador (Nível 2)

- └─ Pode cadastrar atletas em qualquer atlética
- └─ Pode criar, editar e excluir qualquer atlética
- └─ Pode criar e gerenciar times de qualquer atlética
- └─ Pode dar cargo: Capitão
- └─ Não pode remover cargos
- └─ Sem restrição de atlética

Capitão (Nível 1)

- └─ Pode cadastrar atletas APENAS na sua atlética
- └─ Pode criar e gerenciar APENAS a sua atlética
- └─ Pode editar informações da sua atlética
- └─ Pode criar e gerenciar times APENAS da sua atlética
- └─ Pode adicionar/remover atletas dos times da sua atlética

- └─ Não pode dar ou remover cargos
- └─ Restrito à sua própria atlética

Regras de Autenticação

RN-AUTH-01: Login por Matrícula

- Usuário faz login com **matrícula** (não username)
- Sistema busca atleta pela matrícula
- Verifica se atleta tem cargo atribuído
- Valida senha contra hash armazenado
- Carrega informações da atlética vinculada

RN-AUTH-02: Senha Padrão

- Ao atribuir cargo, senha inicial = matrícula do atleta
- Exemplo: Matrícula 2023001 → Senha inicial 2023001

RN-AUTH-03: Primeiro Login

- Sistema detecta `primeiro_login = True`
- Redireciona obrigatoriamente para troca de senha
- Usuário não acessa dashboard até trocar senha
- Após troca, `primeiro_login = False`

RN-AUTH-04: Validação de Senha Forte

- Mínimo 8 caracteres
- Pelo menos 1 letra maiúscula
- Pelo menos 1 letra minúscula
- Pelo menos 1 número
- Não pode ser senha comum (senha123, password123, etc)

Regras de Gestão de Atléticas

RN-ATLETICA-01: Criar Atlética

- **Quem pode:** Admin, Moderador, Capitão
- **Informações obrigatórias:**
 - Nome da atlética (único)
 - Campus
 - Curso
 - Capitão (usuário com cargo Capitão)
- **Validações:**
 - Nome único no sistema
 - Capitão deve ter cargo "Capitão"
 - Capitão pode estar vinculado a apenas uma atlética como responsável

RN-ATLETICA-02: Editar Atlética

- **Admin/Moderador:** Pode editar qualquer atlética
- **Capitão:** Pode editar APENAS a sua própria atlética
- **Campos editáveis:** Nome, campus, curso
- **Não editável:** Capitão (requer remoção e nova atribuição)

RN-ATLETICA-03: Excluir Atlética

- **Quem pode:** Apenas Admin e Moderador
- **Capitão:** Não pode excluir atléticas
- **Validações:**
 - Não pode ter atletas cadastrados
 - Não pode ter times criados
 - Confirmação obrigatória

RN-ATLETICA-04: Vínculo Capitão-Atlética

- Cada Capitão é responsável por UMA atlética
- Ao dar cargo de Capitão, deve-se especificar a atlética
- Capitão só acessa funcionalidades da sua atlética
- Admin/Moderador não têm restrição de atlética

Regras de Gestão de Atletas

RN-ATLETA-01: Cadastrar Atleta

- **Admin/Moderador:** Pode cadastrar em qualquer atlética
- **Capitão:** Pode cadastrar APENAS na sua atlética
- **Validações:**
 - Matrícula: 6-10 dígitos numéricos
 - Data de nascimento: Idade entre 14 e 100 anos
 - Matrícula única no sistema
 - Atlética obrigatória
- **Campos:** Matrícula, nome, data de nascimento, sexo, atlética

RN-ATLETA-02: Editar Atleta

- **Admin/Moderador:** Pode editar qualquer atleta
- **Capitão:** Pode editar APENAS atletas da sua atlética
- **Campos editáveis:** Nome, data de nascimento, sexo
- **Não editáveis:** Matrícula, atlética (requer transferência)

RN-ATLETA-03: Transferir Atleta

- **Quem pode:** Apenas Admin e Moderador
- **Efeito:** Muda atleta de uma atlética para outra
- **Validações:**
 - Atleta não pode estar em times ativos
 - Confirmação obrigatória

Regras de Gestão de Times

RN-TIME-01: Criar Time

- **Admin/Moderador:** Pode criar time para qualquer atlética
- **Capitão:** Pode criar time APENAS para sua atlética
- **Informações obrigatórias:**

- Nome do time
- Modalidade
- Atlética
- Lista de atletas (mínimo conforme modalidade)
- **Validações:**
 - Todos os atletas devem ser da mesma atlética
 - Número de atletas conforme regras da modalidade
 - Atletas não podem estar em outro time da mesma modalidade

RN-TIME-02: Editar Time

- **Admin/Moderador:** Pode editar qualquer time
- **Capitão:** Pode editar APENAS times da sua atlética
- **Operações:**
 - Adicionar atleta (da mesma atlética)
 - Remover atleta
 - Alterar nome do time
- **Validações:**
 - Manter número mínimo de atletas
 - Atletas devem ser da mesma atlética

RN-TIME-03: Excluir Time

- **Admin/Moderador:** Pode excluir qualquer time
- **Capitão:** Pode excluir APENAS times da sua atlética
- **Validações:**
 - Confirmação obrigatória
 - Não pode estar em competição ativa

RN-TIME-04: Modalidades e Regras

- Cada modalidade define:
 - Número mínimo de atletas

- Número máximo de atletas
 - Tipo (individual, dupla, equipe)
- Exemplos:
 - Futsal: 5-12 atletas
 - Vôlei: 6-12 atletas
 - Basquete: 5-12 atletas
 - Xadrez: 1 atleta (individual)

Regras de Permissões

RN-PERM-01: Cadastrar Atleta

- **Admin/Moderador:** Qualquer atlética
- **Capitão:** Apenas sua atlética
- **Validações:**
 - Matrícula: 6-10 dígitos numéricos
 - Data de nascimento: Idade entre 14 e 100 anos
 - Matrícula única
 - Atlética obrigatória

RN-PERM-02: Dar Cargo

- **Admin pode dar:** Capitão (com atlética), Moderador
- **Moderador pode dar:** Capitão (com atlética)
- **Capitão:** Não pode dar cargos
- **Validações:**
 - Atleta deve existir
 - Atleta não pode já ter cargo
 - Email deve ser válido
 - Ao dar cargo Capitão, deve especificar atlética

RN-PERM-03: Remover Cargo

- **Quem pode:** Apenas Admin

- **Efeitos:**
 - Deleta UserProfile
 - Deleta Associado
 - Se for Capitão, libera atlética para novo capitão
 - Atleta permanece no sistema (apenas perde acesso)
- **Confirmação:** Obrigatória via JavaScript

RN-PERM-04: Gestão de Atlética

- **Admin/Moderador:** Acesso total a todas as atléticas
- **Capitão:** Acesso restrito à sua própria atlética
- **Verificação:** Sistema valida `user._profile.id_atletica` antes de permitir ações

RN-PERM-05: Gestão de Times

- **Admin/Moderador:** Pode gerenciar times de qualquer atlética
- **Capitão:** Pode gerenciar APENAS times da sua atlética
- **Verificação:** Sistema valida se `time.atletica == user._profile.id_atletica`

Regras de Dados

RN-DATA-01: Matrícula

- Formato: Apenas números
- Tamanho: 6 a 10 dígitos
- Única no sistema
- Exemplo válido: 2023001 , 20231234567

RN-DATA-02: Email

- Formato: email válido (RFC 5322)
- Único no sistema
- Preferência: @academico.ifpb.edu.br

RN-DATA-03: Data de Nascimento

- Não pode ser futura
- Idade mínima: 14 anos
- Idade máxima: 100 anos

RN-DATA-04: Cargo

- Valores permitidos: Admin , Moderador , Capitao
- Constraint no banco de dados
- Case-sensitive

RN-DATA-05: Nome da Atlético

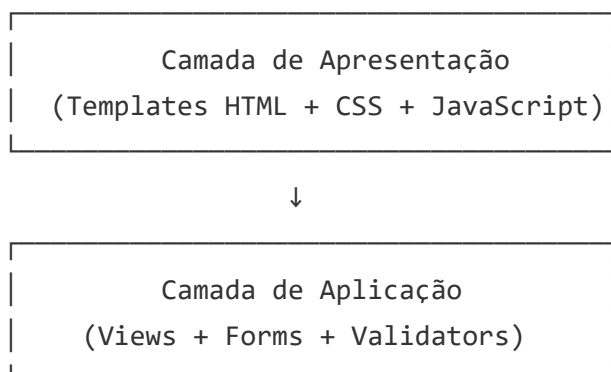
- Único no sistema
- Mínimo 3 caracteres
- Máximo 255 caracteres

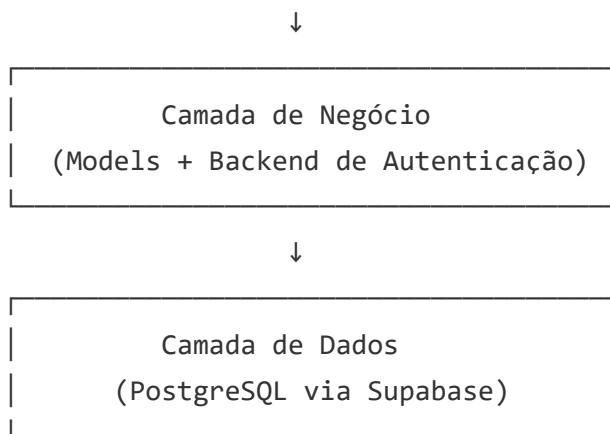
RN-DATA-06: Nome do Time

- Único por modalidade e atlética
- Mínimo 3 caracteres
- Máximo 100 caracteres

Arquitetura do Sistema

Arquitetura em Camadas





Estrutura de Diretórios

```
JES/
├── project/                                # Configurações do Django
│   ├── settings.py                        # Configurações principais
│   └── urls.py                            # URLs raiz
│
├── accounts/                              # App principal
│   ├── models.py                         # Modelos de dados
│   ├── views.py                          # Lógica de views
│   ├── forms.py                          # Formulários
│   ├── urls.py                           # URLs do app
│   ├── validators.py                     # Validadores customizados
│   ├── constants.py                      # Constantes e mensagens
│   └── templates/accounts/               # Templates HTML
│       ├── login.html
│       ├── dashboard.html
│       ├── change_password.html
│       ├── cadastrar_atleta.html
│       ├── dar_cargo.html
│       ├── remover_cargo.html
│       ├── criar_atletica.html
│       ├── editar_atletica.html
│       ├── listar_atleticas.html
│       ├── criar_time.html
│       ├── editar_time.html
│       └── listar_times.html
│
└──
```



- └─ manage.py
- └─ README.md

- # Script de gerenciamento
- # Documentação básica

Padrões de Design Utilizados

MVT (Model-View-Template)

- **Model:** Define estrutura de dados e relacionamentos
- **View:** Lógica de negócio, controle de acesso e validações
- **Template:** Apresentação (HTML) com lógica de exibição condicional

Backend de Autenticação Customizado

- Implementa BaseBackend
- Sobrescreve `authenticate()` e `get_user()`
- Permite login por matrícula
- Carrega informações da atlética vinculada

Form Validation Pattern

- Validação em múltiplas camadas
- `clean_<field>()` : Validação de campo individual
- `clean()` : Validação cross-field e regras de negócio
- Validators reutilizáveis em `validators.py`

Permission Mixin Pattern

- Verificação de permissões nas views
- Validação de vínculo com atlética para Capitães
- Mensagens de erro padronizadas

Modelo de Dados

Diagrama ER

Modalidade
id
nome
tipo
min_atletas
max_atletas

PK



1:N

Atletica
id_atletica
nome
campus
id_curso
capitao_id

PK

UNIQUE

FK

UserProfile
id_usuario
email
password
hash_senha
cargo
primeiro_login
id_atletica

PK

UNIQUE

FK



1:N

Atleta
matricula
nome
data_nasc
sexo
id_atletica

PK

FK



Associado



id_usuario	PK, FK → UserProfile
matricula	FK → Atleta

Time	
id_time	PK
nome	
id_atletica	FK → Atletica
id_modalidade	FK → Modalidade

↑
N:N

TimeAtleta	
id_time	FK → Time
matricula	FK → Atleta

Tabelas Detalhadas

Tabela: atletica

Campo	Tipo	Constraints	Descrição
id_atletica	SERIAL	PK	ID único da atlética
nome	VARCHAR(255)	NOT NULL, UNIQUE	Nome da atlética
campus	VARCHAR(255)	NULL	Campus do IFPB
id_curso	INTEGER	NULL	ID do curso
capitao_id	INTEGER	FK → usuario, NULL	Capitão responsável

Relacionamentos: - 1 Atlética → N Atletas - 1 Atlética → N Times - 1 Atlética → 1 Capitão (UserProfile)

Tabela: atleta

Campo	Tipo	Constraints	Descrição
matricula	VARCHAR(255)	PK	Matrícula do estudante
nome	VARCHAR(255)	NOT NULL	Nome completo
data_nascimento	DATE	NULL	Data de nascimento
sexo	CHAR(1)	NULL	M ou F
id_atletica	INTEGER	FK, NOT NULL	Referência à atlética

Validações: - Matrícula: 6-10 dígitos numéricos - Data de nascimento: Idade 14-100 anos - Atlética obrigatória

Tabela: usuario

Campo	Tipo	Constraints	Descrição
id_usuario	SERIAL	PK	ID único do usuário
email	VARCHAR(254)	UNIQUE, NOT NULL	Email do usuário
password	VARCHAR(255)	NOT NULL	Hash da senha (Django)
hash_senha	VARCHAR(255)	NOT NULL	Hash legado (sincronizado)
cargo	VARCHAR(50)	CHECK, NULL	Admin/Moderador/Capitao
primeiro_login	BOOLEAN	DEFAULT TRUE	Flag de primeiro acesso
id_atletica	INTEGER	FK, NULL	Atlética vinculada (obrigatório para Capitão)

Constraints: - CHECK (cargo IN ('Admin', 'Moderador', 'Capitao')) - CHECK (cargo = 'Capitao' AND id_atletica IS NOT NULL OR cargo != 'Capitao')

Tabela: associado

Campo	Tipo	Constraints	Descrição
id_usuario	INTEGER	PK, FK → usuario	Referência ao usuário
matricula_atleta	VARCHAR(255)	FK → atleta	Referência ao atleta

Propósito: Liga atleta a usuário com cargo

Tabela: modalidade

Campo	Tipo	Constraints	Descrição
id_modalidade	SERIAL	PK	ID único da modalidade
nome	VARCHAR(100)	NOT NULL, UNIQUE	Nome da modalidade
tipo	VARCHAR(20)	NOT NULL	individual/dupla/equipe
min_atletas	INTEGER	NOT NULL	Número mínimo de atletas
max_atletas	INTEGER	NOT NULL	Número máximo de atletas

Exemplos: - Futsal: tipo=equipe, min=5, max=12 - Vôlei: tipo=equipe, min=6, max=12 - Xadrez: tipo=individual, min=1, max=1

Tabela: time

Campo	Tipo	Constraints	Descrição
id_time	SERIAL	PK	ID único do time
nome	VARCHAR(100)	NOT NULL	Nome do time
id_atletica	INTEGER	FK → atletica, NOT NULL	Atlética do time
id_modalidade	INTEGER	FK → modalidade, NOT NULL	Modalidade do time

Constraints: - UNIQUE (nome, id_atletica, id_modalidade) - Nome único por atlética e modalidade

Tabela: time_atleta (Tabela de Relacionamento)

Campo	Tipo	Constraints	Descrição
id_time	INTEGER	FK → time, PK	Referência ao time
matricula_atleta	VARCHAR(255)	FK → atleta, PK	Referência ao atleta

Constraints: - UNIQUE (id_time, matricula_atleta) - Atleta não pode estar duplicado no mesmo time - Validação: Atleta deve ser da mesma atlética do time

Relacionamentos

```
# Atleta → Atlética (N:1)
atleta.id_atletica → atletica.id_atletica

# UserProfile → Atlética (N:1)
usuario.id_atletica → atletica.id_atletica

# Atlética → Capitão (1:1)
atletica.capitao_id → usuario.id_usuario

# Associado → UserProfile (1:1)
associado.id_usuario → usuario.id_usuario

# Associado → Atleta (N:1)
associado.matricula_atleta → atleta.matricula

# Time → Atlética (N:1)
time.id_atletica → atletica.id_atletica

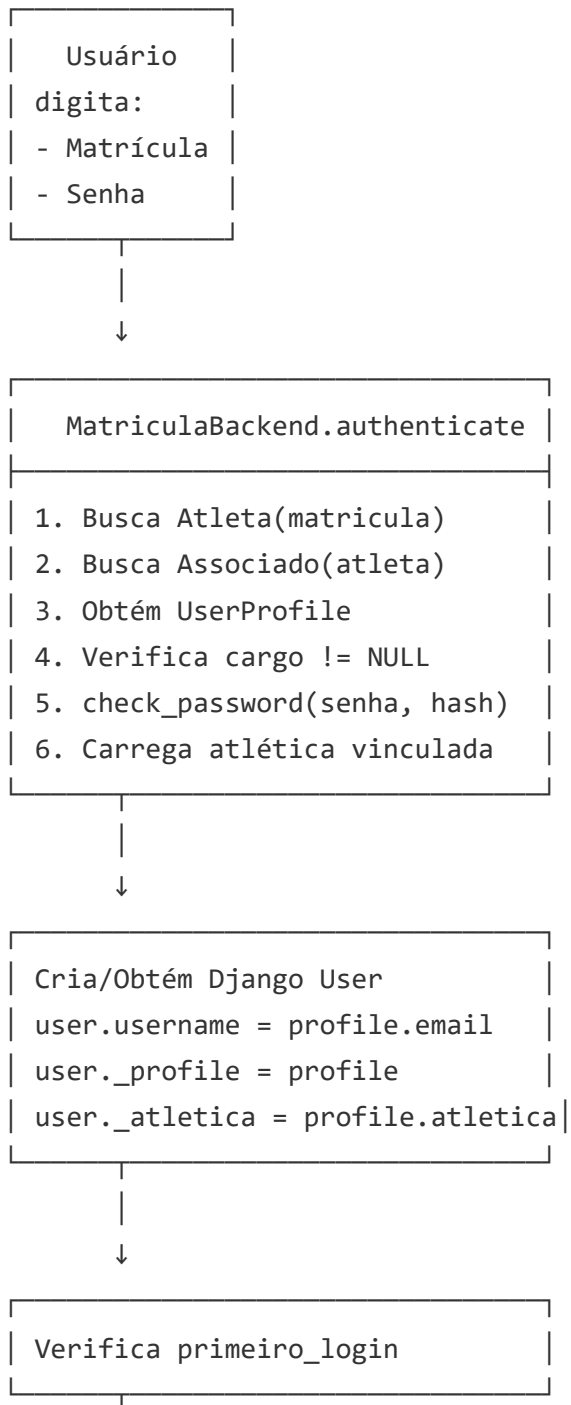
# Time → Modalidade (N:1)
time.id_modalidade → modalidade.id_modalidade

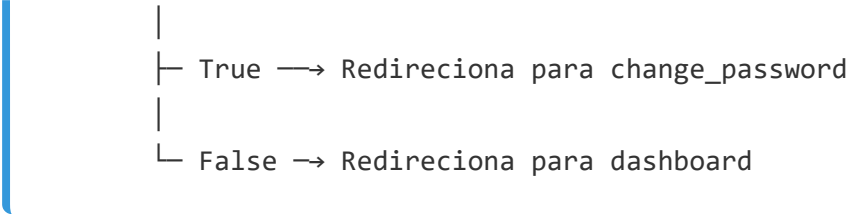
# Time ↔ Atleta (N:N via time_atleta)
```

```
time_atleta.id_time → time.id_time  
time_atleta.matricula_atleta → atleta.matricula
```

Autenticação e Autorização

Fluxo de Autenticação





Matriz de Permissões

Ação	Admin	Moderador	Capitão	Sem Cargo
Login	✓	✓	✓	✗
Cadastrar Atleta	✓ Qualquer	✓ Qualquer	✓ Sua atlética	✗
Criar Atlética	✓	✓	✓	✗
Editar Atlética	✓ Qualquer	✓ Qualquer	✓ Sua atlética	✗
Excluir Atlética	✓	✓	✗	✗
Criar Time	✓ Qualquer	✓ Qualquer	✓ Sua atlética	✗
Editar Time	✓ Qualquer	✓ Qualquer	✓ Sua atlética	✗
Excluir Time	✓ Qualquer	✓ Qualquer	✓ Sua atlética	✗
Dar Cargo (Capitão)	✓	✓	✗	✗
Dar Cargo (Moderador)	✓	✗	✗	✗
Remover Cargo	✓	✗	✗	✗
Ver Estatísticas Globais	✓	✓	✗	✗
Ver Estatísticas da Atlética	✓	✓	✓ Sua atlética	✗
Trocar Senha	✓	✓	✓	✗

Verificação de Permissões por Atlética

Para Capitães, todas as operações verificam:

```
# Verificar se é Capitão e se está acessando sua própria atlética
if user._profile.cargo == 'Capitao':
    if objeto.id_atletica != user._profile.id_atletica:
        # ACESSO NEGADO
        messages.error(request, 'Você só pode gerenciar sua própria atlética')
        return redirect('dashboard')
```

Para Admin e Moderador, não há restrição de atlética.

Funcionalidades

Gestão de Atléticas

Criar Atlética

URL: /accounts/criar-atletica/

Método: GET, POST

Permissão: Admin, Moderador, Capitão

Campos: - Nome da Atlética (CharField, único) - Campus (CharField) - Curso (ForeignKey ou CharField) - Capitão (ForeignKey para UserProfile com cargo=Capitão)

Validações: - Nome único no sistema - Capitão deve ter cargo "Capitão" - Capitão não pode estar vinculado a outra atlética

Fluxo: 1. Usuário acessa formulário 2. Preenche dados da atlética 3. Seleciona capitão (lista de usuários com cargo Capitão sem atlética) 4. Sistema valida e cria atlética 5. Vincula capitão à atlética (`usuario.id_atletica = atletica.id`) 6. Exibe mensagem de sucesso 7. Redireciona para dashboard ou lista de atléticas

Editar Atlética

URL: /accounts/editar-atletica/<id>/

Método: GET, POST

Permissão: - Admin/Moderador: Qualquer atlética - Capitão: Apenas sua atlética

Validação de Acesso: - Se Capitão: `atletica.id == user._profile.id_atletica` -
Senão: Acesso negado

Campos Editáveis: - Nome da atlética - Campus - Curso

Não Editável: Capitão (requer processo de troca)

Listar Atléticas

URL: `/accounts/listar-atleticas/`

Método: GET

Permissão: Admin, Moderador, Capitão

Exibição: - Admin/Moderador: Lista todas as atléticas - Capitão: Lista apenas sua atlética

Informações Exibidas: - Nome da atlética - Campus - Capitão - Número de atletas -
Número de times - Ações: Editar, Ver Detalhes, Excluir (Admin/Moderador)

Excluir Atlética

URL: `/accounts/excluir-atletica/<id>/`

Método: POST

Permissão: Apenas Admin e Moderador

Validações: - Atlética não pode ter atletas cadastrados - Atlética não pode ter times criados - Confirmação JavaScript obrigatória

Efeitos: - Deleta atlética - Libera capitão (remove vínculo)

Gestão de Times

Criar Time

URL: `/accounts/criar-time/`

Método: GET, POST

Permissão: - Admin/Moderador: Qualquer atlética - Capitão: Apenas sua atlética

Campos: - Nome do Time (CharField) - Modalidade (ForeignKey) - Atlética (ForeignKey)
- Admin/Moderador: Pode escolher qualquer - Capitão: Pré-preenchido com sua atlética (readonly) - Atletas (MultipleChoiceField) - Filtrado por atletas da atlética selecionada

Validações: - Nome único por atlética e modalidade - Número de atletas conforme regras da modalidade - Todos os atletas devem ser da mesma atlética - Atletas não podem estar em outro time da mesma modalidade

Fluxo: 1. Usuário acessa formulário 2. Seleciona modalidade 3. Sistema carrega regras (min/max atletas) 4. Seleciona atlética (se Admin/Moderador) 5. Sistema filtra atletas disponíveis da atlética 6. Seleciona atletas (respeitando min/max) 7. Sistema valida e cria time 8. Cria registros em `time_atleta` 9. Exibe mensagem de sucesso

Editar Time

URL: `/accounts/editar-time/<id>/`

Método: GET, POST

Permissão: - Admin/Moderador: Qualquer time - Capitão: Apenas times da sua atlética

Validação de Acesso: - Se Capitão: `time.id_atletica == user._profile.id_atletica`

Operações: - Alterar nome do time - Adicionar atletas (da mesma atlética) - Remover atletas (respeitando mínimo)

Validações: - Manter número mínimo de atletas - Não exceder número máximo - Atletas devem ser da mesma atlética

Listar Times

URL: `/accounts/listar-times/`

Método: GET

Permissão: Admin, Moderador, Capitão

Filtros: - Por modalidade - Por atlética (Admin/Moderador) - Capitão: Vê apenas times da sua atlética

Exibição: - Nome do time - Modalidade - Atlética - Número de atletas - Ações: Ver Detalhes, Editar, Excluir

Excluir Time

URL: `/accounts/excluir-time/<id>/`

Método: POST

Permissão: - Admin/Moderador: Qualquer time - Capitão: Apenas times da sua atlética

Validações: - Confirmação JavaScript - Não pode estar em competição ativa (futura implementação)

Efeitos: - Deleta registros em `time_atleta` - Deleta `time`

Outras Funcionalidades

Login

URL: `/accounts/login/`

Método: GET, POST

Permissão: Pública

Campos: - Matrícula (`CharField`) - Senha (`PasswordInput`)

Validações: - Credenciais válidas - Usuário tem cargo - Usuário ativo

Fluxo: 1. Usuário digita matrícula e senha 2. Sistema autentica via `MatriculaBackend` 3. Se `primeiro_login = True` → Troca de senha 4. Senão → Dashboard

Dashboard

URL: `/accounts/dashboard/`

Método: GET

Permissão: `@login_required`

Exibe: - Informações do usuário (email, cargo, atleta, atlética) - **Admin:** Estatísticas globais (total atletas, usuários, atléticas, times) - **Moderador:** Estatísticas globais - **Capitão:** Estatísticas da sua atlética (atletas, times) - Cards de ações disponíveis (baseado em permissões)

Trocar Senha

URL: `/accounts/change-password/`

Método: GET, POST

Permissão: `@login_required`

Campos: - Senha Atual - Nova Senha (com validação forte) - Confirmar Nova Senha

Validações: - Senha atual correta - Nova senha forte (8+ chars, maiúscula, minúscula, número) - Senhas coincidem - Nova senha $\neq$ senha atual

Cadastrar Atleta

URL: /accounts/cadastrar-atleta/

Método: GET, POST

Permissão: Admin, Moderador, Capitão

Campos: - Matrícula - Nome - Data de Nascimento - Sexo - Atlética -

Admin/Moderador: Pode escolher qualquer - Capitão: Pré-preenchido com sua atlética (readonly)

Validações: - Matrícula única, 6-10 dígitos - Idade 14-100 anos - Atlética obrigatória

Dar Cargo

URL: /accounts/dar-cargo/

Método: GET, POST

Permissão: Admin (Capitão/Moderador), Moderador (Capitão)

Campos: - Matrícula do Atleta - Cargo (opções filtradas por permissão) - Email - Atlética (se cargo = Capitão)

Validações: - Atleta existe - Atleta não tem cargo - Email válido e único - Se Capitão: Atlética obrigatória e sem capitão

Efeitos: - Cria UserProfile (senha = matrícula, primeiro_login = True) - Cria Associado - Se Capitão: Vincula à atlética

Remover Cargo

URL: /accounts/remover-cargo/

Método: GET, POST

Permissão: Apenas Admin

Campos: - Matrícula do Atleta

Validações: - Atleta existe e tem cargo - Confirmação JavaScript

Efeitos: - Deleta Associado - Deleta UserProfile - Se era Capitão: Libera atlética

Fluxos de Uso

Fluxo: Capitão Criando sua Atléticoa

1. Capitão faz login (já tem cargo atribuído por Admin)
↓
2. Acessa "Criar Atléticoa"
↓
3. Preenche:
 - Nome: "Atléticoa IFPB Campina Grande"
 - Campus: "Campina Grande"
 - Curso: "Informática"↓
4. Sistema cria atléticoa e vincula capitão
↓
5. Capitão agora pode:
 - Cadastrar atletas na sua atléticoa
 - Criar times da sua atléticoa

Fluxo: Capitão Criando Time de Futsal

1. Capitão acessa "Criar Time"
↓
2. Seleciona Modalidade: Futsal
 - Sistema mostra: "Mínimo 5, Máximo 12 atletas"↓
3. Atléticoa já vem pré-selecionada (sua atléticoa)
↓
4. Sistema lista apenas atletas da sua atléticoa
↓
5. Capitão seleciona 8 atletas
↓
6. Digita nome: "IFPB CG Futsal Masculino"
↓
7. Sistema valida:
 - 8 atletas está entre 5-12 ✓
 - Todos da mesma atléticoa ✓
 - Nenhum em outro time de futsal ✓

↓

8. Time criado com sucesso

Fluxo: Admin Gerenciando Múltiplas Atléticas

1. Admin faz login

↓

2. Vê dashboard com estatísticas:

- 5 atléticas cadastradas
- 120 atletas total
- 15 times criados

↓

3. Acessa "Listar Atléticas"

- Vê todas as 5 atléticas

↓

4. Clica em "Editar" na Atlética IFPB João Pessoa

↓

5. Altera campus de "JP" para "João Pessoa"

↓

6. Acessa "Criar Time"

↓

7. Pode escolher qualquer atlética

- Seleciona: Atlética IFPB Patos
- Modalidade: Vôlei
- Seleciona atletas de IFPB Patos

↓

8. Time criado para outra atlética (Admin pode)

Fluxo: Moderador Dando Cargo de Capitão

1. Moderador acessa "Dar Cargo"

↓

2. Digita matrícula: 2023005

↓

3. Seleciona cargo: Capitão

↓

4. Sistema exibe campo adicional: "Atlética"

↓

5. Moderador seleciona: "Atlética IFPB Sousa"

- ↓
6. Digita email: capitao@academico.ifpb.edu.br
 - ↓
 7. Sistema valida:
 - Atleta existe ✓
 - Não tem cargo ✓
 - Atlético não tem capitão ✓
 - ↓
 8. Cria UserProfile vinculado à atlética
 - ↓
 9. Informa: "Senha padrão: 2023005"

Fluxo: Capitão Tentando Acessar Outra Atlético

1. Capitão da "Atlético A" faz login
- ↓
2. Tenta acessar URL: /editar-atletica/2/
(ID 2 = Atlético B)
- ↓
3. Sistema verifica:
 - user._profile.cargo = "Capitao"
 - user._profile.id_atletica = 1 (Atlético A)
 - atletica_id = 2 (Atlético B)
 - 1 ≠ 2 → ACESSO NEGADO
- ↓
4. Mensagem: "Você só pode gerenciar sua própria atlética"
- ↓
5. Redireciona para dashboard

Validações e Segurança

Validadores Customizados

Validação de Matrícula

- Apenas números

- 6 a 10 dígitos
- Único no sistema

Validação de Senha Forte

- Mínimo 8 caracteres
- Pelo menos 1 maiúscula
- Pelo menos 1 minúscula
- Pelo menos 1 número
- Não pode ser senha comum

Validação de Data de Nascimento

- Não pode ser futura
- Idade mínima: 14 anos
- Idade máxima: 100 anos

Validação de Nome de Atlético

- Único no sistema
- Mínimo 3 caracteres
- Máximo 255 caracteres

Validação de Time

- Nome único por atlética e modalidade
- Número de atletas conforme modalidade
- Todos atletas da mesma atlética
- Atletas não duplicados em times da mesma modalidade

Segurança Implementada

Proteção contra Acesso Não Autorizado

- Verificação de login em todas as views (@login_required)
- Verificação de cargo antes de permitir ações

- Verificação de vínculo com atlética para Capitães
- Mensagens de erro genéricas (não revelam informações sensíveis)

Proteção contra Brute Force

- Mensagens de erro genéricas no login
- Não revela se matrícula existe
- Mensagem: "Credenciais inválidas"

Hashing de Senhas

- Django `make_password()` e `check_password()`
- Algoritmo: PBKDF2 (padrão Django)
- Salt automático

CSRF Protection

- Token CSRF em todos os formulários
- `{% csrf_token %}` nos templates

SQL Injection Protection

- Django ORM (queries parametrizadas)
- Nenhuma query raw sem sanitização

XSS Protection

- Auto-escaping nos templates Django
- `{{ variable }}` sempre escapado

Validação de Propriedade

- Capitão só acessa recursos da sua atlética
 - Verificação em todas as views de edição/exclusão
 - Validação no backend (não apenas frontend)
-

Configurações

Variáveis de Ambiente Recomendadas

```
DB_NAME=seu_banco
DB_USER=seu_usuario
DB_PASSWORD=sua_senha
DB_HOST=seu_host.supabase.co
DB_PORT=5432

SECRET_KEY=sua_secret_key_aqui
DEBUG=True
```

Configurações de Produção

- DEBUG = False
- ALLOWED_HOSTS configurado
- HTTPS obrigatório
- Cookies seguros
- HSTS habilitado

Dependências Principais

- Django 6.0.2
- psycopg2-binary
- python-decouple (para variáveis de ambiente)

Próximas Funcionalidades (Roadmap)

Gestão de Competições

- Criar competições por modalidade
- Inscrever times em competições
- Gerenciar tabelas e resultados

Sistema de Pontuação

- Pontuação por modalidade
- Ranking de atléticas
- Histórico de resultados

Relatórios e Estatísticas

- Relatórios de participação
- Estatísticas por atlética
- Exportação de dados (PDF, Excel)

Notificações

- Notificações de novas competições
- Lembretes de jogos
- Avisos para capitães

Versão: 2.0

Data: Fevereiro 2026

Sistema: JES - Jogos Escolares Superiores do IFPB